

Configuracion tecnica del servidor de produccion SISA

1. Proposito del documento

Este documento describe de forma tecnica y minuciosa las configuraciones realizadas y recomendadas para poner en produccion el sistema SISA. Incluye la arquitectura del servidor, configuracion de Nginx, PHP-FPM, Laravel, Quasar/Vue, base de datos, colas de trabajo con Supervisor, permisos, certificados SSL, logs, backups, monitoreo y procedimientos de despliegue.

El objetivo es dejar una guia de referencia para administradores de servidor, soporte tecnico y responsables de mantenimiento del sistema.

2. Resumen ejecutivo de la arquitectura

El sistema se desplego como una aplicacion web separada en dos capas principales:

- Frontend: aplicacion Quasar/Vue compilada como SPA y servida por Nginx como archivos

estaticos.

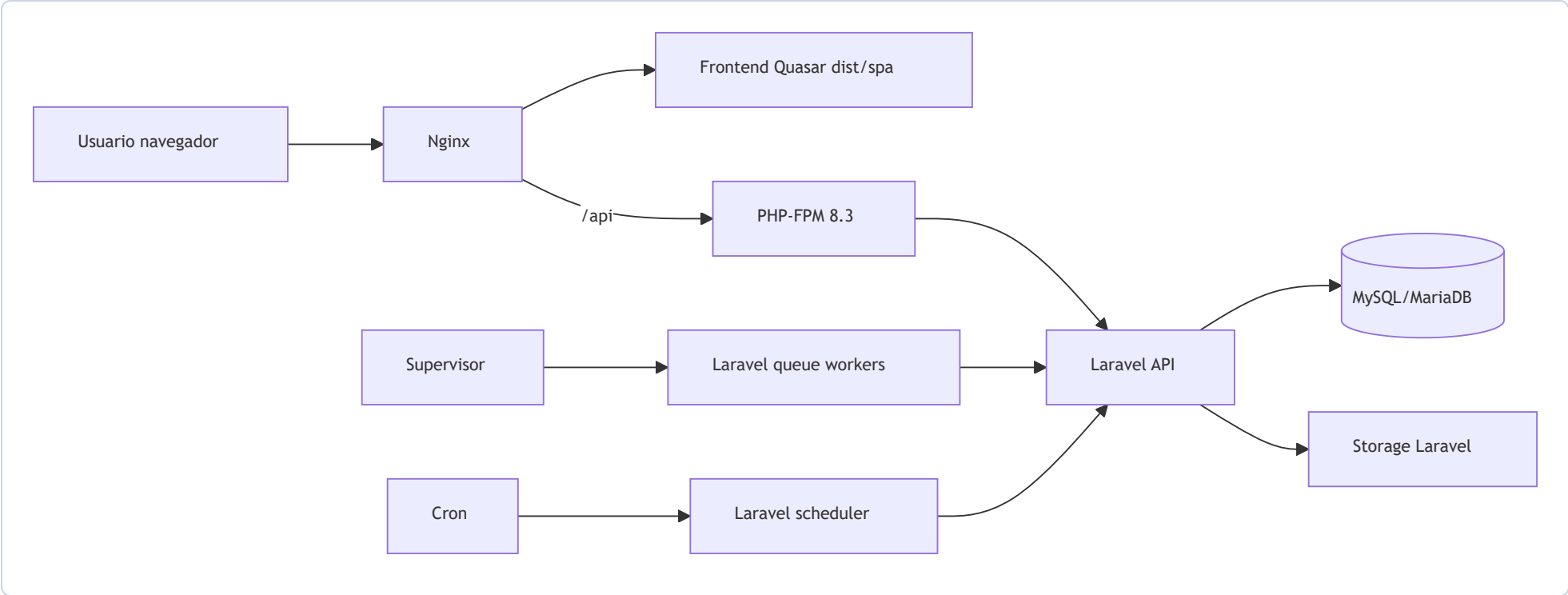
- Backend: aplicacion Laravel expuesta por Nginx mediante PHP-FPM para atender las rutas

`/api`.

Adicionalmente, se utiliza Supervisor para mantener activos los workers de Laravel que procesan tareas en segundo plano. Estas tareas son necesarias para trabajos pesados, procesos diferidos, generacion de documentos, sincronizaciones o cualquier proceso que no debe depender directamente del ciclo de vida de una peticion HTTP.

Componentes principales:

Componente	Uso
Nginx	Servidor web y reverse/front controller para frontend y backend.
PHP-FPM 8.3	Ejecucion del backend Laravel.
Laravel 12	API, autenticacion, reglas de negocio, colas y almacenamiento.
Quasar/Vue 3	Interfaz web del sistema academico.
MySQL/MariaDB	Base de datos principal.
Supervisor	Administracion de workers de Laravel.
Certbot/Let's Encrypt	Certificados SSL para HTTPS.
Cron	Ejecucion del scheduler de Laravel, cuando aplica.



3. Estructura de carpetas en produccion

La estructura usada en el servidor Linux sigue el patron `/var/www`.

Rutas principales identificadas en la configuracion:

```
/var/www/academico-frontend
/var/www/academico-frontend/dist/spa
/var/www/academico-backend
/var/www/academico-backend/public
/var/www/academico-backend/storage/app/public
```

Para una instancia adicional de documentacion o rama separada se definio tambien:

```
/var/www/documentacion.unitepc.edu.bo
/var/www/documentacion.unitepc.edu.bo/Academico
```

```
/var/www/documentacion.unitepc.edu.bo/Academico/dist/spa
/var/www/documentacion.unitepc.edu.bo/back-2file
/var/www/documentacion.unitepc.edu.bo/back-2file/public
/var/www/documentacion.unitepc.edu.bo/back-2file/storage/app/public
```

La separacion entre frontend y backend permite:

- Servir archivos estaticos con mejor rendimiento.
- Mantener el backend protegido detras de PHP-FPM.
- Publicar solo la carpeta `public` de Laravel.
- Evitar exponer `.env`, `storage`, `vendor`, `app` y otros directorios internos.

4. Dominios y endpoints

Configuracion principal:

Dominio / host	Proposito
<code>planificacion.unitepc.edu.bo</code>	Dominio principal del sistema.
<code>documentacion.unitepc.edu.bo</code>	Instancia/documentacion o despliegue alternativo.
<code>192.168.50.22:8000</code>	Configuracion backend interna o de soporte, segun archivo Nginx existente.

Rutas expuestas:

Ruta	Atiende	Descripcion
<code>/</code>	Frontend Quasar	Carga la SPA desde <code>dist/spa/index.html</code> .
<code>/api/...</code>	Backend Laravel	Nginx envia la peticion a <code>public/index.php</code> mediante PHP-FPM.
<code>/storage/...</code>	Archivos publicos Laravel	Archivos de <code>storage/app/public</code> .
<code>/descargas/...</code>	Archivos publicos de descargas	Alias hacia <code>public/descargas</code> , en la instancia que lo usa.

5. Nginx

5.1 Rol de Nginx en produccion

Nginx cumple tres funciones:

1. Servir el frontend compilado de Quasar/Vue.
2. Enviar las peticiones `/api` al backend Laravel por PHP-FPM.
3. Exponer archivos publicos controlados como `/storage` y `/descargas`.

No se usa el servidor de desarrollo de Quasar ni `php artisan serve` en produccion. Nginx es el punto de entrada estable para las peticiones HTTP/HTTPS.

5.2 Configuracion principal para `planificacion.unitepc.edu.bo`

La configuracion versionada se encuentra en:

```
Servidor/nginx/sites-available/sidopa
```

Puntos clave:

```
server {
    listen 80;
    server_name planificacion.unitepc.edu.bo;
    client_max_body_size 10M;

    root /var/www/academico-frontend/dist/spa;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location ~ ^/api(/.*)?$ {
        fastcgi_pass unix:/var/run/php/php8.3-fpm.sock;
        fastcgi_index index.php;
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME /var/www/academico-backend/public/index.php;
        fastcgi_param SCRIPT_NAME /index.php;
        fastcgi_param REQUEST_URI $request_uri;
        fastcgi_param DOCUMENT_ROOT /var/www/academico-backend/public;
    }
}
```

```
location /storage {
    alias /var/www/academico-backend/storage/app/public;
    try_files $uri $uri/ =404;
}

location ~ /\.ht {
    deny all;
}
}
```

Explicacion:

- `root /var/www/academico-frontend/dist/spa` : define la carpeta publica del frontend

compilado.

- `try_files $uri $uri/ /index.html` : permite que Vue Router/Quasar maneje la navegacion

de la SPA. Aunque el frontend usa hash mode, esta regla tambien protege recargas.

- `location ~ ^/api(/.*)?$` : todo lo que empieza con `/api` se procesa por Laravel.
- `fastcgi_pass unix:/var/run/php/php8.3-fpm.sock` : comunica Nginx con PHP-FPM.
- `SCRIPT_FILENAME` : fuerza a que Laravel reciba todas las rutas API por su

`public/index.php` .

- `REQUEST_URI` : conserva la ruta original de la peticion.
- `DOCUMENT_ROOT` : apunta a la carpeta publica de Laravel.
- `location /storage` : permite acceder a archivos subidos o generados que fueron

publicados en `storage/app/public` .

- `location ~ /\.ht` : bloquea archivos ocultos tipo `.htaccess` .

5.3 Configuracion para instancia `documentacion.unitepc.edu.bo`

Archivo:

```
Servidor/nginx/sites-available/documentacion.unitepc.edu.bo
```

Diferencias principales:

- Usa `client_max_body_size 25M` , util para cargas mas grandes.
- Sirve frontend desde `/var/www/documentacion.unitepc.edu.bo/Academico/dist/spa` .
- Envia `/api` hacia `/var/www/documentacion.unitepc.edu.bo/back-2file/public/index.php` .
- Expone `/storage/` y `/descargas/` con `alias` .
- Registra logs propios:
- `/var/log/nginx/documentacion.unitepc.edu.bo.access.log`
- `/var/log/nginx/documentacion.unitepc.edu.bo.error.log`

5.4 Configuracion backend por puerto

Archivo:

```
Servidor/nginx/sites-available/sidopa-backend
```

Esta configuracion escucha en el puerto `8000` y apunta directamente al backend:

```
listen 8000;
server_name 192.168.50.22;
root /var/www/back-2file/public;
```

Uso posible:

- Pruebas internas.
- Diagnostico de backend.
- Exposicion controlada en red privada.

Recomendacion: no exponer este puerto publicamente sin firewall, autenticacion o reglas de red adecuadas.

5.5 Activacion de sitios Nginx

Comandos usados/recomendados:

```
sudo cp Servidor/nginx/sites-available/sidopa /etc/nginx/sites-available/sidopa
sudo ln -s /etc/nginx/sites-available/sidopa /etc/nginx/sites-enabled/sidopa
sudo nginx -t
sudo systemctl reload nginx
```

Para la instancia de documentacion:

```
sudo cp Servidor/nginx/sites-available/documentacion.unitepc.edu.bo /etc/nginx/sites-available/documentacion.unitepc.edu.bo
sudo ln -s /etc/nginx/sites-available/documentacion.unitepc.edu.bo /etc/nginx/sites-enabled/documentacion.unitepc.edu.bo
sudo nginx -t
sudo systemctl reload nginx
```

5.6 Validaciones de Nginx

Comandos utiles:

```
sudo nginx -t
sudo systemctl status nginx
sudo systemctl reload nginx
sudo journalctl -u nginx -n 100 --no-pager
tail -f /var/log/nginx/access.log
tail -f /var/log/nginx/error.log
```

Para logs especificos:

```
tail -f /var/log/nginx/sidopa-backend-access.log
tail -f /var/log/nginx/sidopa-backend-error.log
tail -f /var/log/nginx/documentacion.unitepc.edu.bo.access.log
tail -f /var/log/nginx/documentacion.unitepc.edu.bo.error.log
```

6. PHP-FPM

6.1 Version y socket

El backend Laravel se ejecuta mediante PHP-FPM. Las configuraciones Nginx apuntan al socket:

```
/var/run/php/php8.3-fpm.sock
```

Esto indica que se esta usando PHP 8.3 FPM para procesar Laravel.

6.2 Validaciones

```
php -v
php -m
systemctl status php8.3-fpm
sudo systemctl restart php8.3-fpm
sudo journalctl -u php8.3-fpm -n 100 --no-pager
```

6.3 Extensiones necesarias

Por las dependencias del backend, se requieren extensiones comunes de Laravel y de procesamiento documental:

- pdo_mysql
- mbstring
- openssl
- tokenizer
- xml
- ctype
- json
- fileinfo
- zip
- gd o soporte de imagenes, recomendado por generacion/procesamiento documental.
- curl
- intl, recomendado para fechas y localizacion.

El backend usa paquetes como:

- Laravel Framework 12.
- Laravel Sanctum.
- PhpSpreadsheet para importacion/exportacion Excel.
- PhpWord para documentos Word.
- DomPDF para PDF.
- Smalot PdfParser para lectura de PDF.

7. Backend Laravel

7.1 Ruta del backend

Rutas identificadas:

```
/var/www/academico-backend
/var/www/documentacion.unitepc.edu.bo/back-2file
```

En ambos casos, Nginx solo debe apuntar a:

```
public/index.php
```

Nunca debe exponerse el directorio raiz completo de Laravel.

7.2 Instalacion de dependencias

Comando de produccion:

```
cd /var/www/academico-backend
composer install --no-dev --optimize-autoloader
```

Para una instancia por worktree:

```
cd /var/www/documentacion.unitepc.edu.bo/back-2file
composer install --no-dev --optimize-autoloader
```

Explicacion:

- `--no-dev`: evita instalar herramientas de desarrollo.
- `--optimize-autoloader`: optimiza la carga de clases para produccion.

7.3 Variables de entorno Laravel

El archivo `.env` de produccion debe tener valores reales en el servidor, pero no debe versionarse ni compartirse. En este documento se muestran ejemplos anonimizados.

Variables principales:

```
APP_NAME=Academico
APP_ENV=production
APP_KEY=base64:VALOR_GENERADO
APP_DEBUG=false
APP_URL=https://DOMINIO_DEL_BACKEND_O_SISTEMA
```

```
APP_LOCALE=es
APP_FALLBACK_LOCALE=es
APP_FAKER_LOCALE=es_ES
```

```
LOG_CHANNEL=stack
LOG_STACK=single
LOG_LEVEL=error
```

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=NOMBRE_BASE
DB_USERNAME=USUARIO_BASE
DB_PASSWORD=PASSWORD_SEGURO
```

```
SESSION_DRIVER=database
SESSION_LIFETIME=120
```

```
QUEUE_CONNECTION=database
CACHE_STORE=database
FILESYSTEM_DISK=local
```

```
SANCTUM_STATEFUL_DOMAINS=DOMINIO_FRONTEND
```

Variables de integracion externa:

```
UNIVERSITY_BASE_URL=https://SERVICIO_EXTERNO/api/...
UNIVERSITY_AUTH_URL=https://SERVICIO_SSO/auth/token
UNIVERSITY_CLIENT_ID=CLIENTE
UNIVERSITY_CLIENT_SECRET=SECRETO
```

Importante:

- `APP_DEBUG=false` en produccion.
- `APP_ENV=production`.
- `APP_KEY` debe generarse una sola vez por instancia con `php artisan key:generate`.
- No publicar credenciales reales en repositorios ni documentos.
- `SANCTUM_STATEFUL_DOMAINS` debe coincidir con el dominio del frontend si se usan

cookies/autenticacion stateful.

7.4 Comandos Laravel de despliegue

Comandos aplicados/recomendados:

```
php artisan key:generate
php artisan migrate --force
php artisan storage:link
php artisan optimize:clear
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

Uso de cada comando:

Comando	Funcion
key:generate	Genera clave de aplicacion si no existe.
migrate --force	Ejecuta migraciones en produccion sin confirmacion interactiva.
storage:link	Publica storage/app/public en public/storage .
optimize:clear	Limpia caches previas antes de regenerarlas.
config:cache	Cachea configuracion para mejorar rendimiento.
route:cache	Cachea rutas.
view:cache	Precompila vistas Blade.

7.5 Permisos Laravel

Laravel necesita permisos de escritura en:

```
storage
bootstrap/cache
```

Comandos:

```
sudo chown -R www-data:www-data storage bootstrap/cache
sudo chmod -R ug+rw storage bootstrap/cache
```

Si el despliegue usa otro usuario de sistema, se recomienda agregarlo al grupo `www-data` o ajustar permisos con cuidado.

7.6 Storage publico

El backend usa:

```
storage/app/public
```

Y Laravel publica el enlace:

```
public/storage -> storage/app/public
```

Nginx expone `/storage` usando `alias` . Esto se usa para archivos generados, imagenes, documentos y otros recursos que deben ser descargables.

Validaciones:

```
php artisan storage:link
ls -la public/storage
ls -la storage/app/public
```

8. Frontend Quasar/Vue

8.1 Ruta del frontend

Rutas identificadas:

```
/var/www/academico-frontend
/var/www/academico-frontend/dist/spa
/var/www/documentacion.unitepc.edu.bo/Academico
/var/www/documentacion.unitepc.edu.bo/Academico/dist/spa
```

8.2 Build de produccion

Comandos:

```
cd /var/www/academico-frontend
npm ci
npm run build
```

O para la instancia con carpeta `Academico`:

```
cd /var/www/documentacion.unitepc.edu.bo/Academico
npm ci
npm run build
```

Resultado esperado:

```
dist/spa
```

Nginx sirve esa carpeta directamente.

8.3 Consideraciones del frontend

El proyecto Quasar esta configurado como SPA con:

```
vueRouterMode: 'hash'
```

Esto permite compatibilidad con hosting estatico, ya que las rutas del frontend usan hash. Aun asi, Nginx mantiene `try_files ... /index.html` para que la SPA cargue correctamente.

8.4 Variables de entorno frontend

La API en produccion debe apuntar al dominio correcto. En la implementacion se preparo el frontend para trabajar con `/api`, evitando depender de dominios externos fijos.

Ejemplo:

```
VITE_API_URL=/api
PUBLIC_BASE_URL=https://planificacion.unitepc.edu.bo
```

Si se usa una instancia distinta:

```
VITE_API_URL=/api
PUBLIC_BASE_URL=https://documentacion.unitepc.edu.bo
```

9. Base de datos

9.1 Motor

El backend esta configurado para usar:

```
DB_CONNECTION=mysql
```

Esto implica MySQL o MariaDB.

9.2 Creacion de base y usuario

Ejemplo anonimo:

```
CREATE DATABASE sidopa CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'sidopa_user'@'localhost' IDENTIFIED BY 'PASSWORD_SEGURO';
GRANT ALL PRIVILEGES ON sidopa.* TO 'sidopa_user'@'localhost';
FLUSH PRIVILEGES;
```

Para una instancia separada:

```
CREATE DATABASE sidopa_documentacion CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'sidopa_documentacion_user'@'localhost' IDENTIFIED BY 'PASSWORD_SEGURO';
GRANT ALL PRIVILEGES ON sidopa_documentacion.* TO 'sidopa_documentacion_user'@'localhost';
FLUSH PRIVILEGES;
```

9.3 Migraciones

```
php artisan migrate --force
php artisan migrate:status
```

9.4 Copia inicial desde otra base

Si se requiere partir de datos existentes:

```
mysqldump -u root -p base_actual > /tmp/sidopa_actual.sql
mysql -u root -p sidopa_documentacion < /tmp/sidopa_actual.sql
```

Recomendaciones:

- Hacer backup antes de importar.
- Confirmar que la base destino sea la correcta.
- No mezclar produccion y pruebas en la misma base.
- Usar usuarios con privilegios limitados para la aplicacion.

10. Colas de trabajo y Supervisor

10.1 Por que se usa Supervisor

Laravel puede ejecutar tareas en segundo plano mediante colas. En produccion, no se debe mantener un worker abierto manualmente por consola. Para esto se usa Supervisor, que:

- Inicia workers automaticamente al arrancar el servidor.
- Reinicia workers si fallan.
- Mantiene varios procesos concurrentes.
- Centraliza logs de salida y errores.
- Permite reiniciar workers despues de cada despliegue.

En el sistema, los workers son importantes para procesos como:

- Generacion de documentos.
- Procesamiento de archivos Excel.
- Generacion de reportes pesados.
- Sincronizaciones academicas.
- Tareas diferidas de backend.
- Cualquier job Laravel que use `QUEUE_CONNECTION`.

10.2 Driver de cola

La configuracion de Laravel usa:

```
QUEUE_CONNECTION=database
```

El archivo `config/queue.php` confirma el driver `database` como conexion por defecto. Esto significa que los jobs se almacenan en la tabla `jobs` y los fallidos en `failed_jobs`.

Tablas requeridas:

```
jobs
job_batches
failed_jobs
```

Si no existen, se generan con migraciones Laravel correspondientes.

10.3 Configuracion recomendada de Supervisor

Archivo sugerido:

```
/etc/supervisor/conf.d/sidopa-worker.conf
```

Contenido recomendado:

```
[program:sidopa-worker]
process_name=%(program_name)s_%(process_num)02d
command=php /var/www/academico-backend/artisan queue:work database --sleep=3 --tries=3 --timeout=120 --max-time=3600
autostart=true
autorestart=true
stopasgroup=true
killasgroup=true
user=www-data
numprocs=2
redirect_stderr=true
stdout_logfile=/var/log/supervisor/sidopa-worker.log
stopwaitsecs=3600
```

Explicacion de parametros:

Parametro	Funcion
command	Comando que ejecuta el worker Laravel.
queue:work database	Procesa la cola almacenada en base de datos.
--sleep=3	Espera 3 segundos cuando no hay trabajos.
--tries=3	Intenta cada job hasta 3 veces antes de marcarlo fallido.
--timeout=120	Tiempo maximo por job. Ajustar si se generan documentos pesados.
--max-time=3600	Recicla el worker cada hora para liberar memoria.
numprocs=2	Ejecuta 2 workers en paralelo.
user=www-data	Ejecuta con el usuario del servidor web.
autorestart=true	Reinicia el proceso si falla.
stopasgroup/killasgroup	Detiene procesos hijos correctamente.
stdout_logfile	Guarda logs del worker.

Para instancia documentacion.unitepc.edu.bo :

```
[program:sidopa-documentacion-worker]
process_name=%(program_name)s_%(process_num)02d
command=php /var/www/documentacion.unitepc.edu.bo/back-2file/artisan queue:work database --sleep=3 --tries=3 --timeout=120 --max-time=3600
autostart=true
autorestart=true
stopasgroup=true
killasgroup=true
user=www-data
numprocs=2
redirect_stderr=true
stdout_logfile=/var/log/supervisor/sidopa-documentacion-worker.log
stopwaitsecs=3600
```

10.4 Activar configuracion de Supervisor

```
sudo supervisorctl reread
sudo supervisorctl update
sudo supervisorctl start sidopa-worker:*
sudo supervisorctl status
```

Reiniciar workers despues de un despliegue:

```
php artisan queue:restart
sudo supervisorctl restart sidopa-worker:*
```

Logs:

```
tail -f /var/log/supervisor/sidopa-worker.log
sudo supervisorctl tail -f sidopa-worker:*
```

10.5 Control de jobs fallidos

Comandos utiles:

```
php artisan queue:failed
php artisan queue:retry all
php artisan queue:flush
```

Usar queue:flush solo cuando se confirme que los trabajos fallidos ya no deben reintentarse.

11. Scheduler de Laravel con Cron

Si el sistema tiene tareas programadas en Laravel, se debe registrar el scheduler en cron.

Entrada recomendada:

```
* * * * * cd /var/www/academico-backend && php artisan schedule:run >> /dev/null 2>&1
```

Para instancia alterna:

```
* * * * * cd /var/www/documentacion.unitepc.edu.bo/back-2file && php artisan schedule:run >> /dev/null 2>&1
```

Validacion:

```
crontab -l
php artisan schedule:list
php artisan schedule:run
```

12. SSL y HTTPS

12.1 Certbot

Se recomienda usar Certbot con Nginx:

```
sudo certbot --nginx -d planificacion.unitepc.edu.bo
```

Para instancia de documentacion:

```
sudo certbot --nginx -d documentacion.unitepc.edu.bo
```

12.2 Renovacion

Certbot normalmente instala un timer systemd. Validar:

```
systemctl list-timers | grep certbot
sudo certbot renew --dry-run
```

12.3 Recomendaciones SSL

- Redirigir HTTP a HTTPS.
- Mantener certificados renovados automaticamente.
- Revisar que `APP_URL` use `https`.
- Verificar que el frontend apunte a `/api` o al dominio HTTPS correcto.

13. Firewall y puertos

Puertos recomendados:

Puerto	Uso	Exposicion
80	HTTP / validacion Certbot	Publico, redirigido a HTTPS.
443	HTTPS	Publico.
22	SSH	Restringido por IP si es posible.
3306	MySQL/MariaDB	Solo local o red privada.
8000	Backend interno opcional	No exponer publicamente sin control.

Ejemplo con UFW:

```
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
sudo ufw enable
sudo ufw status verbose
```

Si se usa el puerto `8000`, limitarlo:

```
sudo ufw allow from 192.168.50.0/24 to any port 8000 proto tcp
```

14. Logs

14.1 Logs Nginx

```
/var/log/nginx/access.log
/var/log/nginx/error.log
/var/log/nginx/sidopa-backend-access.log
```

```
/var/log/nginx/sidopa-backend-error.log
/var/log/nginx/documentacion.unitepc.edu.bo.access.log
/var/log/nginx/documentacion.unitepc.edu.bo.error.log
```

14.2 Logs Laravel

```
storage/logs/laravel.log
```

Comando:

```
tail -f storage/logs/laravel.log
```

14.3 Logs Supervisor

```
/var/log/supervisor/sidopa-worker.log
/var/log/supervisor/sidopa-documentacion-worker.log
```

14.4 Logs del sistema

```
journalctl -u nginx -n 100 --no-pager
journalctl -u php8.3-fpm -n 100 --no-pager
systemctl status supervisor
```

15. Backups

15.1 Backup de base de datos

Comando manual:

```
mysqldump -u USUARIO -p NOMBRE_BASE > /var/backups/sidopa/sidopa_$(date +%F_%H%M).sql
```

Comprimido:

```
mysqldump -u USUARIO -p NOMBRE_BASE | gzip > /var/backups/sidopa/sidopa_$(date +%F_%H%M).sql.gz
```

15.2 Backup de archivos

Respaldar:

```
storage/app/public
public/descargas
.env
```

Ejemplo:

```
tar -czf /var/backups/sidopa/storage_$(date +%F_%H%M).tar.gz /var/www/academico-backend/storage/app/public
```

15.3 Recomendaciones de backup

- Mantener backups diarios de base de datos.
- Mantener backups de archivos publicos generados.
- Guardar una copia fuera del servidor.
- Probar restauracion periodicamente.
- Proteger `.env` y dumps SQL porque contienen informacion sensible.

16. Procedimiento de despliegue completo

16.1 Backend

```
cd /var/www/academico-backend
git pull
composer install --no-dev --optimize-autoloader
php artisan down
php artisan migrate --force
php artisan optimize:clear
php artisan config:cache
php artisan route:cache
```

```
php artisan view:cache
php artisan storage:link
php artisan queue:restart
php artisan up
```

Reiniciar servicios:

```
sudo systemctl reload php8.3-fpm
sudo supervisorctl restart sidopa-worker:*
sudo systemctl reload nginx
```

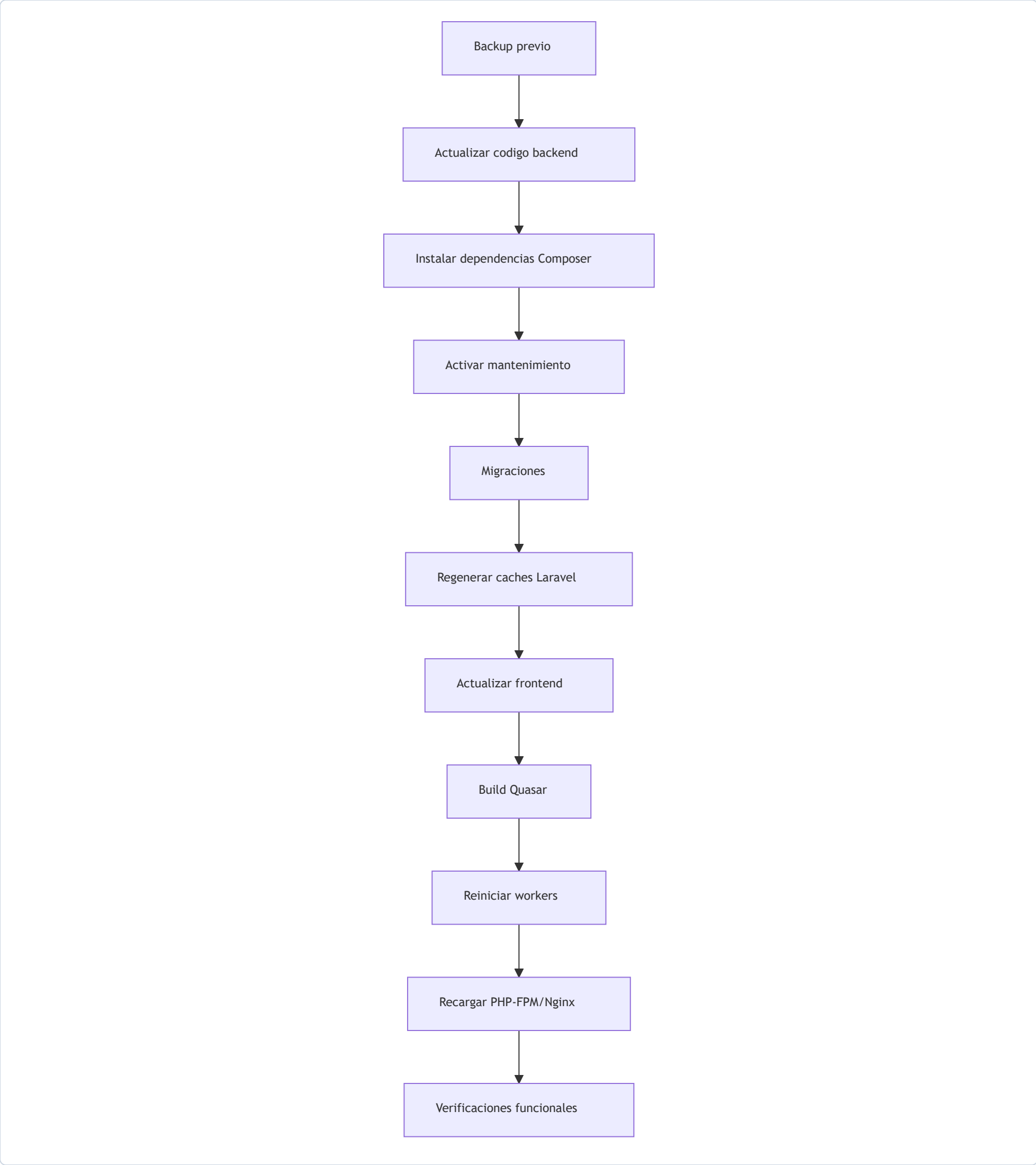
16.2 Frontend

```
cd /var/www/academico-frontend
git pull
npm ci
npm run build
```

Nginx no requiere reinicio si solo cambio el contenido de `dist/spa` , pero se puede recargar si tambien cambio configuracion:

```
sudo nginx -t
sudo systemctl reload nginx
```

16.3 Orden recomendado



17. Rollback

Si un despliegue falla:

- 1. Activar mantenimiento:

```
php artisan down
```

- 1. Volver al commit anterior:

```
git log --oneline -5
git checkout COMMIT_ANTERIOR
```

- 1. Reinstalar dependencias si cambio `composer.lock` :

```
composer install --no-dev --optimize-autoloader
```

1. Regenerar caches:

```
php artisan optimize:clear
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

1. Reconstruir frontend si corresponde:

```
npm ci
npm run build
```

1. Reiniciar workers y levantar:

```
php artisan queue:restart
sudo supervisorctl restart sidopa-worker:*
php artisan up
```

Nota: si hubo migraciones destructivas, el rollback requiere restaurar backup de base de datos.

18. Seguridad aplicada y recomendada

18.1 Backend

- `APP_DEBUG=false`.
- `.env` no debe exponerse ni versionarse.
- Nginx apunta solo a `public/index.php`.
- Se bloquean archivos ocultos con `location ~ /\.(!well-known).*`.
- Permisos restringidos a `storage` y `bootstrap/cache`.
- Credenciales de base de datos con usuario dedicado.

18.2 Frontend

- El frontend usa `/api` para evitar exponer endpoints internos innecesarios.
- No incluir secretos en variables del frontend.
- Recompilar despues de cambiar variables de build.

18.3 Servidor

- SSH restringido.
- Firewall activo.
- Certificados SSL vigentes.
- Logs revisados.
- Backups fuera del servidor.
- Supervisor ejecutando workers como `www-data`, no como `root`.

19. Checklist de verificacion post despliegue

19.1 Servicios

```
systemctl status nginx
systemctl status php8.3-fpm
systemctl status supervisor
supervisorctl status
```

19.2 Laravel

```
php artisan about
php artisan migrate:status
php artisan route:list
php artisan queue:failed
```

19.3 Web

Verificar:

- Carga del dominio principal.
- Login.
- Navegacion por menus.
- Consumo de `/api`.
- Carga de archivos `/storage`.
- Generacion o descarga de documentos.
- Reporte de evaluaciones.

- Banco de preguntas.
- Rol de exámenes.

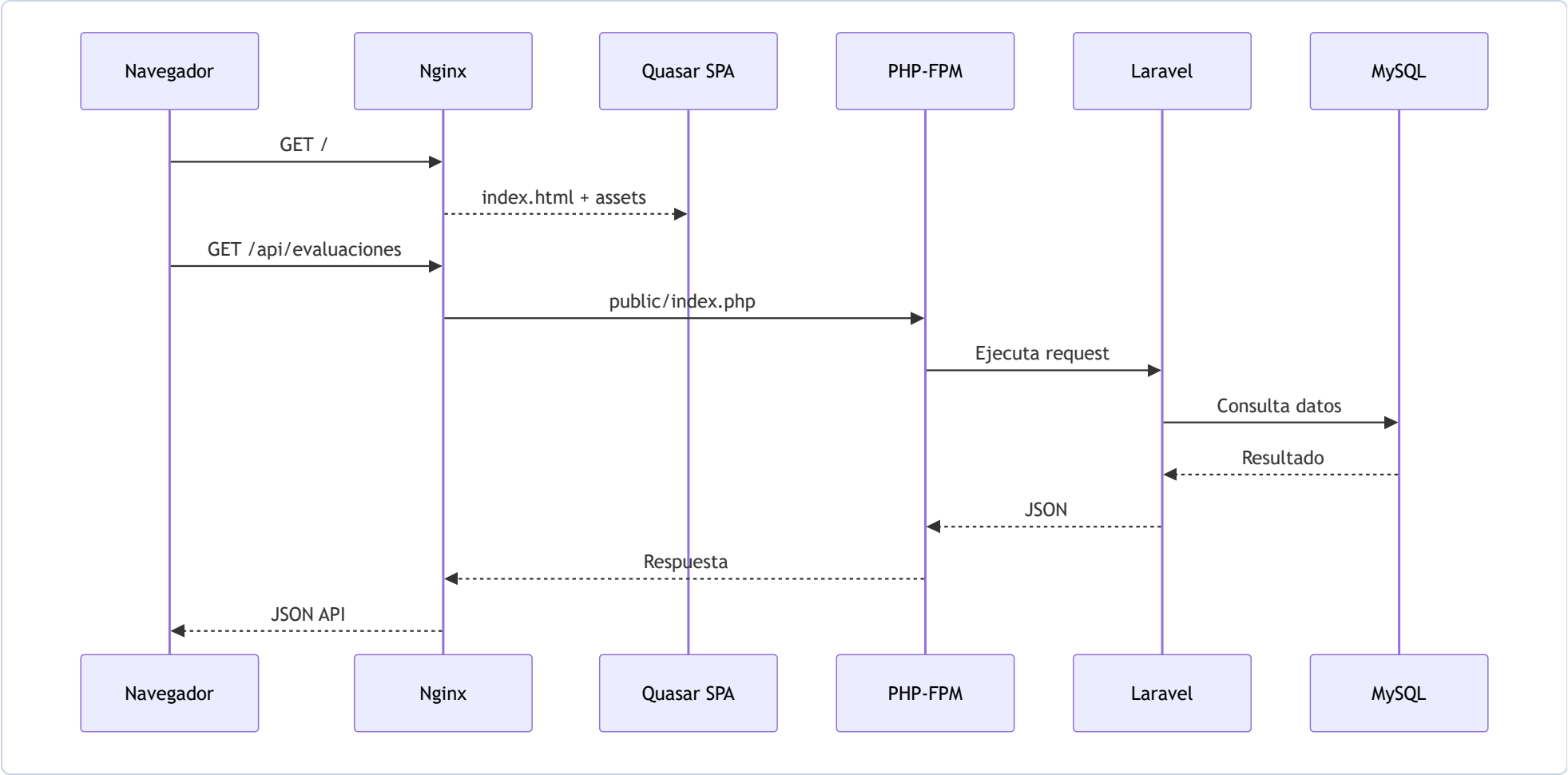
19.4 Logs

```
tail -n 100 storage/logs/laravel.log
tail -n 100 /var/log/nginx/error.log
tail -n 100 /var/log/supervisor/sidopa-worker.log
```

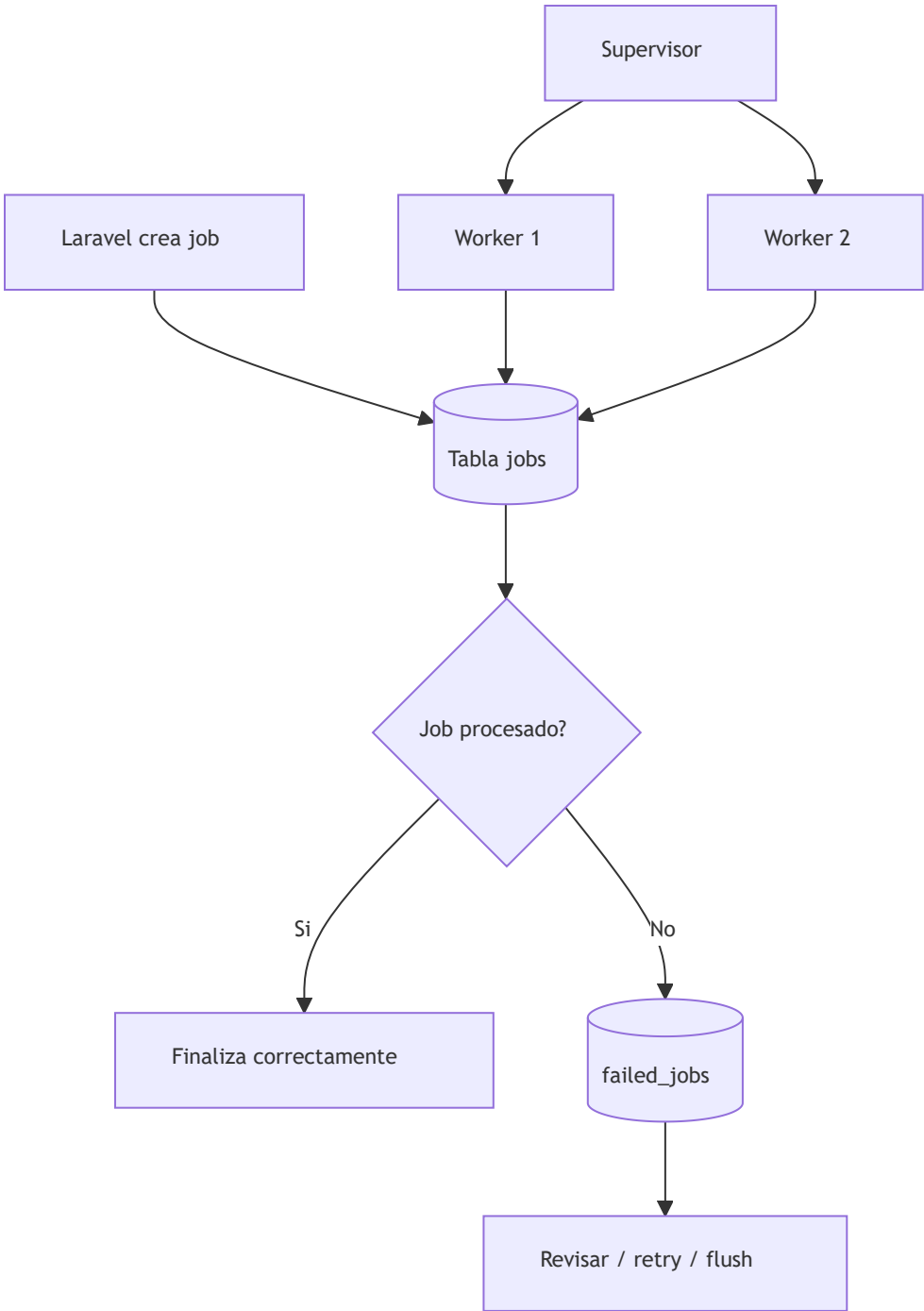
20. Problemas frecuentes y solucion

Problema	Causa probable	Solucion
El frontend carga pero API falla	Nginx no envia <code>/api</code> a Laravel	Revisar bloque <code>location ~ ^/api</code> .
Error 500 en API	Error Laravel o <code>.env</code> incorrecto	Revisar <code>storage/logs/laravel.log</code> .
Error 404 al recargar rutas	SPA sin <code>try_files</code>	Revisar <code>try_files \$uri \$uri/ /index.html</code> .
Archivos <code>/storage</code> no cargan	Falta <code>storage:link</code> o alias Nginx	Ejecutar <code>php artisan storage:link</code> y revisar alias.
Jobs no se procesan	Supervisor detenido o worker caido	<code>supervisorctl status</code> y reiniciar workers.
Cambios <code>.env</code> no aplican	Config cacheada	<code>php artisan optimize:clear</code> && <code>php artisan config:cache</code> .
Permiso denegado en logs/cache	Permisos incorrectos	<code>chown/chmod</code> en <code>storage</code> y <code>bootstrap/cache</code> .
Certificado vencido	Renovacion Certbot fallida	<code>certbot renew --dry-run</code> y revisar timer.
Subida de archivos falla	<code>client_max_body_size</code> bajo	Ajustar Nginx a 25M o segun necesidad.

21. Diagrama de peticion HTTP



22. Diagrama de workers



23. Archivos de configuracion importantes

Archivo	Proposito
<code>Servidor/nginx/sites-available/sidopa</code>	Nginx principal para <code>planificacion.unitepc.edu.bo</code> .
<code>Servidor/nginx/sites-available/planificacion</code>	Variante de configuracion Nginx para dominio principal.
<code>Servidor/nginx/sites-available/sidopa-backend</code>	Backend por puerto <code>8000</code> .
<code>Servidor/nginx/sites-available/documentacion.unitepc.edu.bo</code>	Instancia alterna/documentacion.
<code>back-2file/.env.production</code>	Ejemplo real local de entorno produccion, no compartir secretos.
<code>back-2file/config/queue.php</code>	Configuracion de colas Laravel.
<code>back-2file/config/filesystems.php</code>	Configuracion de almacenamiento.
<code>Academico/quasar.config.js</code>	Configuracion de build frontend.
<code>Academico/package.json</code>	Scripts y dependencias frontend.
<code>back-2file/composer.json</code>	Dependencias backend Laravel.

24. Notas finales

La puesta en produccion de SISA se basa en una arquitectura clasica y estable:

- Nginx sirve el frontend y enruta `/api`.
- Laravel se ejecuta por PHP-FPM.
- MySQL/MariaDB almacena datos.
- Supervisor mantiene activos los workers de cola.
- Cron ejecuta tareas programadas.
- Certbot gestiona HTTPS.

Esta separacion permite que el sistema sea mantenible, escalable y facil de auditar. Para futuras mejoras se recomienda agregar monitoreo centralizado, alertas por caida de workers, rotacion controlada de logs, backups automaticos verificados y un pipeline de despliegue que ejecute build, migraciones, reinicio de workers y pruebas de salud de forma ordenada.